

Scopes

Diseño e Implementación de un Lenguaje

v0.1.0

1. Introducción **1**

1. Introducción

El ejemplo de la sección anterior manipula una tabla sin considerar los *scopes* de las variables involucradas, con lo cual todas las variables son de alcance **global**. Nótese que el scope viene dado por el contenido de la tabla de símbolos en un momento determinado. Si se modifica el programa anterior por:

```
00 int x;
01 float y = 3.14f;
02 for (unsigned int i = 0; i < 5; ++i) {
03     signed int x = 2 * i;
04     y += x;
05 }
06 double z = 48 * u;
07 char x = '\t';
```

En este caso, solo se modifica la línea `03` para que *x* sea declarado dentro del bucle *for*, lo cual producirá un error si se utiliza una sola tabla de símbolos como antes, ya que la variable es de carácter global, incluso ahí dentro.

Para elevar la potencia de la tabla de símbolos, sería útil considerar que en realidad la declaración de la variable *x* en la línea `03` crea una nueva variable, y que esta es **local** al bucle.

Es notorio ver que como el alcance está atado al contenido de la tabla de símbolos, la creación de una nueva tabla de símbolos dentro del bucle o la partición de la tabla original,

permite generar un nuevo alcance sin **colisiones**. ¿En qué momento se debería introducir la nueva tabla de símbolos? Cuando cambie el alcance, y en un programa de este tipo (organizado por bloques de código), el alcance se ve afectado al ingresar y salir de un bloque encerrado entre `{ … }`.

De este modo, cuando el lexer perciba el token **{** creará una nueva tabla de símbolos, y cuando perciba el token **}** destruirá la última **tabla activa**. Este proceso es **anidable** (por ejemplo, dentro del bucle *for*, podría existir otro), con lo cual se debe respetar el hecho de que el ingreso a un nuevo bloque, y por lo tanto a un alcance, debe permitir la visibilidad de las nuevas variables, así como también de aquellas que se encuentren fuera del nuevo alcance, con cierto orden de precedencia.

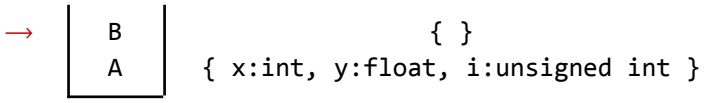
Por ejemplo, luego de analizar la línea `03` por completo, se obtendrán 2 alcances, A y B:

Scope	Id	Type	Initialization
A	x	int	<empty>
A	y	float	3.14
A	i	unsigned int	0
B	x	signed int	<empty>

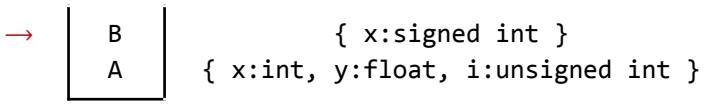
Ahora bien, hay dos entradas para el ID *x*, aunque en diferentes alcances. Al analizar la línea `04`, ¿qué entrada debería utilizarse para el lexema *x*? Debería utilizarse la del alcance B, ya que es el alcance activo, pero... ¿cómo se determina este alcance activo de forma sistemática a lo largo de todo el AST?

Nótese que el alcance se crea al ingresar un bloque, y que deja de tener efecto al salir de él. Es decir, la tabla se expande y contrae a medida que se anidan los bloques, con lo cual, sería útil considerar una estructura de datos de tipo **stack** para gestionar el alcance actual.

Inicialmente, la pila de alcances se encuentra vacía. Luego de ingresar la primera entrada en la tabla para el lexema *x*, se crea un nuevo alcance A y se ingresa a la pila. Al ver el token **{** mediante el lexer, se debe registrar un nuevo alcance, con lo cual el estado de la pila debe ser:



A la derecha de la pila, se indica el conjunto de variables que cada alcance posee declaradas y disponibles para su uso. Luego de ver la línea `03`, el estado de la pila se modifica:



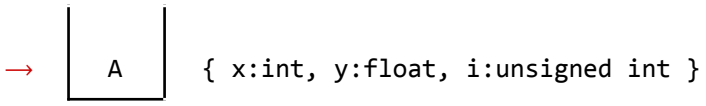
Como se puede observar, se ingresa una nueva instancia para la variable *x*, y de diferente tipo. Al evaluar la sentencia de la línea 04, se hace un uso híbrido de los alcances, ya que *x* pertenece al alcance B, mientras que *y*, al alcance A. Para lograr que el compilador utilice la

versión correcta de cada variable, la búsqueda (el método de consulta de una variable en la tabla de símbolos), debe utilizar la pila, realizando una búsqueda desde el tope de la misma.

Para la variable *x*, se comienza buscando en el tope, y por lo tanto en el alcance B. Este alcance posee una entrada para *x* de tipo *signed int*, con lo cual emplea la misma. Luego, al ver el lexema *y* se procede de la misma manera, pero como el alcance B no posee una entrada, se debe continuar con el siguiente en la pila, que en este caso es A. Este sí posee una entrada para *y* de tipo *float*, y es la versión que finalmente se utiliza como era deseado originalmente.

¿Qué hubiese sucedido si el scope A tampoco tuviese una entrada para *y*? En ese caso la variable no habría sido declarada en un scope accesible, y por lo tanto sería inalcanzable, es decir, se debería emitir un error de compilación.

¿Qué ocurre al parsear la línea `05`? En ese caso, el scope B se cierra, y por lo tanto deja de ser accesible, lo que implica que la pila debe eliminar su tope:



Ahora el nuevo tope es A, y por lo tanto la antigua variable *x* cambia de scope para referenciar a aquella que fue declarada en la línea `01`. Nótese que la variable *x* dentro del bucle *for* pierde su alcance, y por lo tanto se convierte en inaccesible.

En definitiva, el uso de la pila al recorrer el AST permite identificar sobre cada uno de sus nodos cual es el scope activo, dentro de una tabla de símbolos (que en este caso fue particionada mediante una clave adicional que representa el scope, pero que también podría conformarse mediante tablas separadas, una por cada alcance).